

SLIDING WINDOW

DSA PATTERN

From first principles to advanced counting, monotonic structures, proofs, and interview mastery

A one-stop Go reference and learning path

Concepts | Recognition | Templates | Dry runs | Correctness | Pitfalls | 30 practice problems

How to use this guide

Read Chapters 1-5 in order if you are new. Then learn the specialized data structures in Chapters 6-8. Use the template catalog and decision tree while solving. Finish with the graded practice set and solutions.

Contents and learning roadmap

Three-stage roadmap

Basic: fixed windows and incremental state. **Intermediate:** variable windows, frequency maps, counting. **Advanced:** deques, ordered structures, transformations, proofs, and knowing when the pattern is invalid.

A one-stop Go reference and learning path	1
Contents and learning roadmap	2
Go for DSA - a bridge from C++	6
What changes when you move from C++ to Go?	6
The Go rules that matter most for sliding windows	6
A complete first Go window program	6
Reading Go algorithm code quickly	7
1. The core idea	8
1.1 From brute force to reuse	8
1.2 What makes sliding possible?	8
1.3 Complexity intuition: why nested loops can still be $O(n)$	9
2. Recognition and classification	10
2.1 Recognition signals	10
2.2 The main families	10
2.3 The four design questions	10
2.4 A decision checklist	11
3. Fixed-size windows - the foundation	12
3.1 Maximum sum of any length-k subarray	12
3.2 A universal fixed-window template	12
3.3 First negative number in every window	13
3.4 Count distinct values in each window	14
3.5 Rolling product, average, and other aggregates	14
4. Variable windows - longest and shortest	15
4.1 Longest subarray with sum at most S (non-negative values)	15
4.2 Why while, not if?	15
4.3 Minimum length with sum at least S (positive values)	15
4.4 Max consecutive ones after flipping at most k zeros	16

4.5 A subtle lazy-shrink variant	17
5. Strings and frequency-state windows	18
5.1 Longest substring without repeating characters	18
5.2 Find all anagrams of a pattern	19
5.3 Minimum window substring	19
5.4 Replacement windows: longest repeating character	20
6. Counting subarrays correctly	22
6.1 Count ranges with an at-most constraint	22
6.2 Exactly $k = \text{atMost}(k) - \text{atMost}(k - 1)$	22
6.3 Binary arrays with sum exactly goal	23
6.4 Product less than k (positive values)	24
6.5 A general counting theorem	24
7. Monotonic dequeues for window extrema	25
7.1 Sliding window maximum	25
7.2 The two deque invariants	25
7.3 Longest subarray with $\max - \min \leq \text{limit}$	25
7.4 Deque versus heap versus ordered structures	26
8. Ordered windows, medians, and richer state	27
8.1 Fixed-window median with two heaps	27
8.2 Choosing the state representation	29
8.3 Coordinate compression	29
9. When ordinary sliding window fails	30
9.1 Negative numbers break sum monotonicity	30
9.2 Exact sum with arbitrary integers: prefix frequencies	30
9.3 Shortest sum at least K with arbitrary integers	30
9.4 Other warning signs	31
9.5 Sliding window versus related techniques	31
10. Proofs, invariants, and derivation	32
10.1 A proof recipe for variable windows	32
10.2 Proof sketch: longest at most k distinct	32
10.3 Proof sketch: monotonic maximum deque	32
10.4 Derive rather than memorize	32
11. Debugging and common mistakes	33
11.1 Off-by-one rules	33

11.2 Frequent implementation failures	33
11.3 Trace table method	33
11.4 Edge-case test suite	34
12. Go template catalog	35
12.1 Fixed-size template	35
12.2 Longest valid template	35
12.3 Shortest valid template	36
12.4 Count all hereditary-valid windows	36
12.5 Exactly-k transform	36
12.6 Monotonic deque template	37
12.7 Binary search on answer plus fixed window	38
13. Advanced patterns and extensions	39
13.1 Complement window: take from both ends	39
13.2 Circular arrays	39
13.3 Multiple constraints	39
13.4 Two windows in one pass for exactly-k counting	39
13.5 Frequency of the maximum element	39
13.6 Windows over streams	39
13.7 2D sliding windows	39
13.8 Smallest range covering k sorted lists	40
13.9 Offline queries and Mo's algorithm	40
14. Worked pattern map	41
14.1 Interview communication script	41
15. Graded practice set	42
15.1 Basic	42
15.2 Intermediate	42
15.3 Advanced	43
15.4 Expert extensions	43
16. Hints and compact solutions	44
16.1 Problems 1-6	44
16.2 Problems 7-15	44
16.3 Problems 16-25	44
16.4 Problems 26-30	44
16.5 A reusable brute-force oracle	44

17. Final cheat sheet	46
Recognition	46
Core formulas	46
Answer-event map	46
Complexity	46
Mastery test	47
Seven-day mastery plan	47
Before submitting any window solution	47

Go for DSA - a bridge from C++

Go has fewer containers and less syntax than C++, which makes the algorithm visible once a few language conventions become familiar.

What changes when you move from C++ to Go?

C++ idea	Go equivalent	DSA note
vector<int>	[]int	append(s, x); len(s); s[i]
unordered_map<K,V>	map[K]V	Missing keys return the zero value
array<int,256>	[256]int	Zero-initialized automatically
deque<int>	[]int used as a deque	Append at back; reslice or use a head index
priority_queue	container/heap	Implement five interface methods
long long	int64	Convert explicitly: int64(x)
pair / tuple	small struct	Named fields improve readability
exceptions	error return or panic	Validate k at API boundaries

The Go rules that matter most for sliding windows

- **Arrays and slices differ:** [256]int has fixed length; []int is a dynamic view backed by an array.
- **Maps return zero for missing keys:** freq[x]++ works immediately. Use value, found := m[key] when presence itself matters, and delete(m, key) to remove a key.
- **Strings are byte sequences:** s[i] is a byte. Use []rune(s) for Unicode code points. Most interview problems specify ASCII or English letters.
- **No ternary operator:** use a normal if statement. Go favors explicit control flow.
- **Unused imports and variables are compile errors:** this catches clutter but surprises new Go users.
- **int is machine-sized:** indices naturally use int; large sums and counts should use int64 with explicit conversions.

A complete first Go window program

Every listing in this edition follows the same executable shape: package declaration, imports, algorithm function, and main with a small example. Save a listing as main.go and run go run main.go.

```
package main

import "fmt"

func main() {
    nums := []int{2, 1, 5, 1, 3, 2} // slice: similar to vector<int>
    freq := make(map[int]int)       // hash map: similar to unordered_map
    window := 3
    sum := 0

    for right, value := range nums {
        sum += value
        freq[value]++
        if right >= window {
            outgoing := nums[right-window]
            sum -= outgoing
            freq[outgoing]--
            if freq[outgoing] == 0 {
                delete(freq, outgoing)
            }
        }
        if right >= window-1 {
            left := right - window + 1
            fmt.Printf("[%d,%d] sum=%d distinct=%d\n",
                left, right, sum, len(freq))
        }
    }
}
```

Reading Go algorithm code quickly

for right, value := range nums yields an index and a copy of the value. := declares variables using inferred types. Multiple assignment such as left, sum := 0, 0 is common. Slices use half-open bounds: s[a:b] includes a and excludes b. Functions may return multiple values, which is useful for returning an answer plus an error.

C++ habit to unlearn

Do not search for a Go STL equivalent for every C++ container. For sliding windows, slices, arrays, maps, and container/heap cover nearly everything. A slice of indices is the usual deque representation.

1. The core idea

A sliding window reuses the work done for one contiguous range when moving to a nearby range.

1.1 From brute force to reuse

Many array and string problems ask about a **contiguous** subarray or substring. A brute-force solution chooses every left endpoint, every right endpoint, and often scans the range again. That can cost $O(n^3)$, or $O(n^2)$ if each range statistic is maintained incrementally.

A sliding window stores exactly the information needed about one active range **[left, right]**. When *right* advances, one element enters. When *left* advances, one element leaves. If each update is cheap, all useful ranges can often be processed in $O(n)$.



Active window [1, 3] contains {1, 5, 1}; indices appear below.

Vocabulary

Term	Meaning
Window	The current contiguous range [left, right].
Expand	Move right forward and add a new element to the state.
Shrink	Remove a[left], then move left forward.
State	Sum, counts, distinct count, deque, multiset, or other summary.
Invariant	A condition guaranteed at a specific point in the loop.
Valid window	A range satisfying the problem constraint.

1.2 What makes sliding possible?

- **Contiguity:** the object is a subarray or substring, not an arbitrary subsequence.
- **Local update:** adding or removing one boundary item updates the state cheaply.
- **Useful movement rule:** we know when and why to move left or right.
- **Monotonic feasibility for variable windows:** after a violation, removing items from the left moves toward validity and discarded starts never need to return.

Important distinction

Sliding window is a specialized form of the two-pointer technique. All sliding windows use boundaries, but not every two-pointer algorithm maintains a contiguous interval. Pair-sum on a sorted array is two pointers, not usually called sliding window.

1.3 Complexity intuition: why nested loops can still be $O(n)$

Variable-window code often contains a for loop and an inner while. It is not automatically $O(n^2)$. The right pointer moves forward at most n times. The left pointer also moves forward at most n times. No pointer moves backward, so there are at most $2n$ boundary movements. This is aggregate or amortized analysis: $O(n)$ movements total.

The total complexity is $O(n * U)$, where U is the cost of one state update. An array counter or hash map is expected $O(1)$; a balanced tree or multiset makes the same traversal $O(n \log n)$.

2. Recognition and classification

Before writing code, identify the window type, validity rule, state, and answer event.

2.1 Recognition signals

- The statement says subarray, substring, contiguous segment, consecutive, or every block of size k .
- It asks for a longest, shortest, maximum, minimum, number of ranges, or whether any range satisfies a condition.
- Neighboring candidate ranges overlap heavily.
- The constraint can be repaired by moving the left boundary forward.

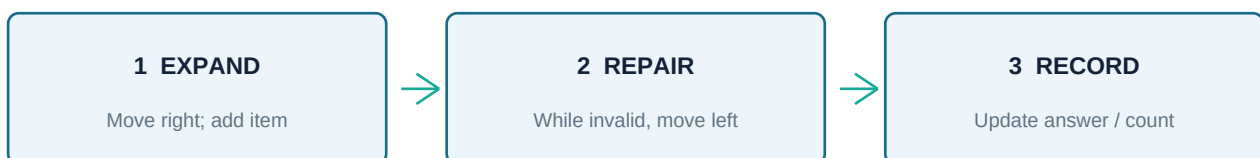
These are clues, not proof. Always test whether discarding a left endpoint is safe. Negative values, exact-equality constraints, or non-monotone properties can invalidate the ordinary variable-window method.

2.2 The main families

Family	Window size	Typical question	Core state
Fixed	Exactly k	Best/each block of length k	Sum or frequency
Variable: longest	Changes	Longest range satisfying at most...	Validity state
Variable: shortest	Changes	Shortest range reaching at least...	Validity state
Counting	Changes	How many ranges satisfy...	At-most invariant
Frequency matching	Fixed or variable	Permutation/anagram/cover	Counts + mismatch
Order statistic	Usually fixed	Maximum, minimum, median	Deque/tree/heaps

2.3 The four design questions

- 1. What does [left, right] mean?** State whether the interval is inclusive. Then length is $\text{right} - \text{left} + 1$.
- 2. What state summarizes it?** Examples: sum, number of zeros, character frequencies, distinct values, maximum-minimum, matched requirements.
- 3. What is invalid?** Write a Boolean predicate such as $\text{sum} > \text{limit}$, $\text{distinct} > k$, or $\text{max} - \text{min} > \text{limit}$.
- 4. When is the answer recorded?** After reaching size k ? After restoring validity? During shrinking? The location of this line is algorithmically important.



The core variable-window control loop

2.4 A decision checklist

Question	If yes	If no
Must the range length equal k ?	Use fixed window.	Consider variable window.
Can invalidity be repaired by removing left items?	Use expand/repair.	Seek prefix sums, DP, binary search, etc.
Are all valid suffixes ending at right countable?	Add window length.	Use another counting transform.
Need max/min under deletions?	Monotonic deque.	Simple scalar state may work.
Negative values affect a sum constraint?	Be suspicious.	Classic sum window may be valid.

3. Fixed-size windows - the foundation

Fixed windows have one deterministic movement: add the new item and remove the item that is now too old.

3.1 Maximum sum of any length-k subarray

Example: [2, 1, 5, 1, 3, 2], $k = 3$. The length-3 sums are 8, 7, 9, and 6. Instead of recomputing each sum, subtract the outgoing element and add the incoming element.

```
package main

import (
    "errors"
    "fmt"
)

func maxSumK(a []int, k int) (int64, error) {
    if k <= 0 || k > len(a) {
        return 0, errors.New("invalid k")
    }
    var sum int64
    for i := 0; i < k; i++ {
        sum += int64(a[i])
    }
    answer := sum
    for right := k; right < len(a); right++ {
        sum += int64(a[right])
        sum -= int64(a[right-k])
        if sum > answer {
            answer = sum
        }
    }
    return answer, nil
}

func main() {
    answer, err := maxSumK([]int{2, 1, 5, 1, 3, 2}, 3)
    if err != nil {
        panic(err)
    }
    fmt.Println(answer) // 9
}
```

Invariant: immediately before updating the answer, sum equals the sum of exactly the k elements ending at right. Time $O(n)$, extra space $O(1)$. Use int64 when sums may exceed the machine-sized int range.

3.2 A universal fixed-window template

```

package main

import "fmt"

func windowSums(a []int, k int) []int {
    if k <= 0 || k > len(a) {
        return nil
    }
    answer := []int{}
    left, sum := 0, 0
    for right, value := range a {
        sum += value // add incoming item
        if right-left+1 > k {
            sum -= a[left] // remove outgoing item
            left++
        }
        if right-left+1 == k {
            answer = append(answer, sum)
        }
    }
    return answer
}

func main() {
    fmt.Println(windowSums([]int{2, 1, 5, 1, 3, 2}, 3))
}

```

This formulation is safer than manually computing outgoing indices and generalizes immediately from sums to maps, mismatch counters, and dequeues.

3.3 First negative number in every window

Maintain a deque of **indices** of negative values. Indices reveal whether an item has expired; storing only values cannot distinguish duplicates or age.

```

package main

import "fmt"

func firstNegative(a []int, k int) []int {
    if k <= 0 || k > len(a) {
        return nil
    }
    queue := []int{} // indices of negative values
    answer := []int{}
    for right, value := range a {
        if value < 0 {
            queue = append(queue, right)
        }
        left := right - k + 1
        for len(queue) > 0 && queue[0] < left {
            queue = queue[1:]
        }
        if left >= 0 {
            if len(queue) == 0 {
                answer = append(answer, 0)
            } else {
                answer = append(answer, a[queue[0]])
            }
        }
    }
    return answer
}

func main() {
    a := []int{12, -1, -7, 8, -15, 30, 16, 28}
    fmt.Println(firstNegative(a, 3))
}

```

3.4 Count distinct values in each window

```
package main

import "fmt"

func distinctPerWindow(a []int, k int) []int {
    if k <= 0 || k > len(a) {
        return nil
    }
    freq := make(map[int]int)
    answer := []int{}
    for right, value := range a {
        freq[value]++
        if right >= k {
            outgoing := a[right-k]
            freq[outgoing]--
            if freq[outgoing] == 0 {
                delete(freq, outgoing)
            }
        }
        if right >= k-1 {
            answer = append(answer, len(freq))
        }
    }
    return answer
}

func main() {
    fmt.Println(distinctPerWindow([]int{1, 2, 1, 3, 4, 2, 3}, 4))
}
```

Map hygiene

Erase zero-frequency keys when map size represents the distinct count. A key with value zero still contributes to `len(freq)` in Go.

3.5 Rolling product, average, and other aggregates

Sums, counts, and XOR are easy to add and remove. Products require care: division fails with zeros, overflow is common, and modular division may not exist. An average is simply sum / k ; keep the sum in an exact integer type and convert when producing the result. Some statistics, such as a maximum, cannot be maintained with one scalar because the outgoing maximum may expose an unknown second-best value. That motivates deques and trees.

4. Variable windows - longest and shortest

The right edge explores. The left edge repairs or optimizes.

4.1 Longest subarray with sum at most S (non-negative values)

With non-negative elements, expanding can only increase the sum and shrinking can only decrease it. This monotonic behavior makes discarded starts safe.

```
package main

import "fmt"

func longestSumAtMost(a []int, limit int64) int {
    left, answer := 0, 0
    var sum int64
    for right, value := range a {
        sum += int64(value)
        for left <= right && sum > limit {
            sum -= int64(a[left])
            left++
        }
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    // Correct because all values are non-negative.
    fmt.Println(longestSumAtMost([]int{2, 1, 5, 1, 1}, 7))
}
```

Loop invariant after the while loop: [left, right] is valid, and every earlier left endpoint has been ruled out for this and all future right endpoints under the non-negative assumption. Therefore the current valid window is the longest valid one ending at right.

4.2 Why while, not if?

One incoming element may require removing several old elements. An if removes at most one, potentially leaving the window invalid. Use while (invalid) whenever validity is not restored by exactly one guaranteed removal.

4.3 Minimum length with sum at least S (positive values)

For a shortest-window problem, record an answer **while the window is valid** and keep shrinking to find a smaller valid range ending at the same right.

```

package main

import "fmt"

func minLengthAtLeast(a []int, target int64) int {
    left, answer := 0, len(a)+1
    var sum int64
    for right, value := range a {
        sum += int64(value)
        for sum >= target {
            if length := right - left + 1; length < answer {
                answer = length
            }
            sum -= int64(a[left])
            left++
        }
    }
    if answer == len(a)+1 {
        return 0
    }
    return answer
}

func main() {
    fmt.Println(minLengthAtLeast([]int{2, 3, 1, 2, 4, 3}, 7))
}

```

Answer placement

Longest under an upper bound: repair invalidity, then record. Shortest reaching a lower bound: record while valid, then shrink. Memorizing the code is less reliable than understanding this event.

4.4 Max consecutive ones after flipping at most k zeros

```

package main

import "fmt"

func longestOnes(a []int, k int) int {
    left, zeros, answer := 0, 0, 0
    for right, value := range a {
        if value == 0 {
            zeros++
        }
        for zeros > k {
            if a[left] == 0 {
                zeros--
            }
            left++
        }
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    a := []int{1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0}
    fmt.Println(longestOnes(a, 2))
}

```

The abstract pattern is not about zeros. It is: maintain a budget of at most k violations. The same idea solves longest substring after at most k replacements, at most k odd numbers, and at most k distinct categories.

4.5 A subtle lazy-shrink variant

Some longest-window solutions replace `while` with `if` and allow the maintained interval to remain notionally invalid, preserving only its maximum length. This can be correct for specific problems such as character replacement, but it changes the invariant and is easy to misuse. Prefer the fully valid template until you can prove the lazy version.

5. Strings and frequency-state windows

Most string windows are boundary movement plus a carefully designed frequency invariant.

5.1 Longest substring without repeating characters

Frequency-count solution

```
package main

import "fmt"

func longestUniqueBytes(s string) int {
    var freq [256]int
    left, answer := 0, 0
    for right := 0; right < len(s); right++ {
        incoming := s[right]
        freq[incoming]++
        for freq[incoming] > 1 {
            freq[s[left]]--
            left++
        }
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    fmt.Println(longestUniqueBytes("abcabcbb")) // ASCII/byte version
}
```

Indexing a Go string returns bytes. A [256]int frequency array is ideal for ASCII problems. For Unicode, convert once with []rune(s) and use a map[rune]int; byte positions and rune positions are not interchangeable.

Last-seen jump optimization

```
package main

import "fmt"

func longestUniqueRunes(s string) int {
    chars := []rune(s) // Unicode code points
    last := make(map[rune]int)
    left, answer := 0, 0
    for right, char := range chars {
        if index, found := last[char]; found && index >= left {
            left = index + 1
        }
        last[char] = right
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    fmt.Println(longestUniqueRunes("aabca"))
}
```

The The index >= left test is essential. A character last seen before the current window must not pull left backward.

5.2 Find all anagrams of a pattern

Every answer has fixed length $|p|$. Maintain a difference count. The missing value counts how many required character instances remain, not merely how many character types remain.

```
package main

import "fmt"

func findAnagrams(s, pattern string) []int {
    if len(pattern) == 0 || len(pattern) > len(s) {
        return nil
    }
    var need [256]int
    for i := 0; i < len(pattern); i++ {
        need[pattern[i]]++
    }
    missing, left := len(pattern), 0
    answer := []int{}
    for right := 0; right < len(s); right++ {
        incoming := s[right]
        if need[incoming] > 0 {
            missing--
        }
        need[incoming]--
        if right-left+1 > len(pattern) {
            outgoing := s[left]
            left++
            need[outgoing]++
            if need[outgoing] > 0 {
                missing++
            }
        }
        if missing == 0 {
            answer = append(answer, left)
        }
    }
    return answer
}

func main() {
    fmt.Println(findAnagrams("cbaebabacd", "abc"))
}
```

Here negative need[c] means the window contains extra copies. When an outgoing character makes need[out] positive, a required instance has become missing again.

5.3 Minimum window substring

This is a variable window with multiplicities. Expand until all required instances are covered. Then shrink while coverage remains complete, recording each improved minimum.

```

package main

import "fmt"

func minWindow(s, target string) string {
    if len(target) == 0 || len(target) > len(s) {
        return ""
    }
    var need [256]int
    for i := 0; i < len(target); i++ {
        need[target[i]]++
    }
    missing, left := len(target), 0
    bestLeft, bestLen := 0, len(s)+1
    for right := 0; right < len(s); right++ {
        incoming := s[right]
        if need[incoming] > 0 {
            missing--
        }
        need[incoming]--
        for missing == 0 {
            if length := right - left + 1; length < bestLen {
                bestLeft, bestLen = left, length
            }
            outgoing := s[left]
            left++
            need[outgoing]++
            if need[outgoing] > 0 {
                missing++
            }
        }
    }
    if bestLen == len(s)+1 {
        return ""
    }
    return s[bestLeft : bestLeft+bestLen]
}

func main() {
    fmt.Println(minWindow("ADOBECODEBANC", "ABC"))
}

```

5.4 Replacement windows: longest repeating character

Let maxFreq be the largest frequency of any character in the window. The number of replacements needed is $\text{windowLength} - \text{maxFreq}$. If this exceeds k , shrink.

```
package main

import "fmt"

func characterReplacement(s string, k int) int {
    var freq [26]int
    left, maxFreq, answer := 0, 0, 0
    for right := 0; right < len(s); right++ {
        index := s[right] - 'A'
        freq[index]++
        if freq[index] > maxFreq {
            maxFreq = freq[index]
        }
        for right-left+1-maxFreq > k {
            freq[s[left]-'A']--
            left++
        }
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    fmt.Println(characterReplacement("AABABBA", 1))
}
```

maxFreq is intentionally not decreased during shrinking. It may be stale, but it is an upper bound representing a frequency achieved by some window of this size. This suffices for the maximum-length result. If you need the exact validity of the current window or its boundaries, recompute/maintain the true maximum instead.

6. Counting subarrays correctly

Counting windows is where a one-line combinatorial observation often replaces an extra loop.

6.1 Count ranges with an at-most constraint

After repairing the window, suppose $[left, right]$ is valid and the property is hereditary under removing items from the left. Then every suffix ending at $right$ is also valid: $[left, right]$, $[left+1, right]$, ..., $[right, right]$. There are exactly $right - left + 1$ of them.

```
package main

import "fmt"

func countAtMostKDistinct(a []int, k int) int64 {
    if k < 0 {
        return 0
    }
    freq := make(map[int]int)
    left, distinct := 0, 0
    var answer int64
    for right, value := range a {
        if freq[value] == 0 {
            distinct++
        }
        freq[value]++
        for distinct > k {
            outgoing := a[left]
            freq[outgoing]--
            if freq[outgoing] == 0 {
                delete(freq, outgoing)
                distinct--
            }
            left++
        }
        answer += int64(right - left + 1)
    }
    return answer
}

func main() {
    fmt.Println(countAtMostKDistinct([]int{1, 2, 1, 2, 3}, 2))
}
```

Counts can reach $n(n+1)/2$, so return `int64` even when indices use `int`.

6.2 Exactly $k = \text{atMost}(k) - \text{atMost}(k - 1)$

The sets of ranges with at most k distinct values and at most $k-1$ distinct values differ by exactly the ranges with k distinct values.

```

package main

import "fmt"

func atMostDistinct(a []int, k int) int64 {
    if k < 0 {
        return 0
    }
    freq := make(map[int]int)
    left := 0
    var answer int64
    for right, value := range a {
        freq[value]++
        for len(freq) > k {
            outgoing := a[left]
            freq[outgoing]--
            if freq[outgoing] == 0 {
                delete(freq, outgoing)
            }
            left++
        }
        answer += int64(right - left + 1)
    }
    return answer
}

func exactlyKDistinct(a []int, k int) int64 {
    return atMostDistinct(a, k) - atMostDistinct(a, k-1)
}

func main() {
    fmt.Println(exactlyKDistinct([]int{1, 2, 1, 2, 3}, 2))
}

```

This transformation also works for exactly k odd values, exactly k zeros, exactly k violations, and other integer-valued metrics whose at-most counts are easy to compute.

6.3 Binary arrays with sum exactly goal

For non-negative/binary arrays, use $\text{count}(\text{sum} == \text{goal}) = \text{atMost}(\text{goal}) - \text{atMost}(\text{goal} - 1)$.

```

package main

import "fmt"

func atMostSum(a []int, goal int) int64 {
    if goal < 0 {
        return 0
    }
    left, sum := 0, 0
    var answer int64
    for right, value := range a {
        sum += value
        for sum > goal {
            sum -= a[left]
            left++
        }
        answer += int64(right - left + 1)
    }
    return answer
}

func countBinarySum(a []int, goal int) int64 {
    return atMostSum(a, goal) - atMostSum(a, goal-1)
}

func main() {
    fmt.Println(countBinarySum([]int{1, 0, 1, 0, 1}, 2))
}

```

6.4 Product less than k (positive values)

```
package main

import "fmt"

func productLessThanK(a []int, k int64) int64 {
    if k <= 1 {
        return 0
    }
    product := int64(1)
    left := 0
    var answer int64
    for right, value := range a {
        product *= int64(value)
        for product >= k {
            product /= int64(a[left])
            left++
        }
        answer += int64(right - left + 1)
    }
    return answer
}

func main() {
    fmt.Println(productLessThanK([]int{10, 5, 2, 6}, 100))
}
```

This assumes positive values. Zeros, negatives, and overflow change the reasoning. Even with positive integers, ensure multiplication fits the chosen type or use a problem-specific overflow guard.

6.5 A general counting theorem

Suffix-count theorem

If, after repair, left is the smallest start such that [left, right] is valid, and validity is preserved when removing elements from the left, then exactly right-left+1 valid subarrays end at right. Summing this over all right endpoints counts each valid subarray once.

7. Monotonic deques for window extrema

A deque compresses all candidates that might become the maximum or minimum of a future window.

7.1 Sliding window maximum

Maintain indices in decreasing order of their values. Before inserting a new value, remove smaller or equal values from the back: the newcomer is at least as good and expires later. Remove expired indices from the front. The front is always the maximum.

```
package main

import "fmt"

func maxSlidingWindow(a []int, k int) []int {
    if k <= 0 || k > len(a) {
        return nil
    }
    deque := []int{} // indices; values decrease front to back
    answer := []int{}
    for right, value := range a {
        for len(deque) > 0 && a[deque[len(deque)-1]] <= value {
            deque = deque[:len(deque)-1]
        }
        deque = append(deque, right)
        left := right - k + 1
        if deque[0] < left {
            deque = deque[1:]
        }
        if left >= 0 {
            answer = append(answer, a[deque[0]])
        }
    }
    return answer
}

func main() {
    a := []int{1, 3, -1, -3, 5, 3, 6, 7}
    fmt.Println(maxSlidingWindow(a, 3))
}
```

7.2 The two deque invariants

- **Index order:** indices increase from front to back. Therefore expiration happens only at the front.
- **Value order:** values decrease from front to back. Therefore the front is the maximum.
- **Dominance:** an older value no larger than a new value can never win in a future window, so it is permanently removed.

Each index is inserted once and removed at most once, giving $O(n)$ time and $O(k)$ space. For a minimum, reverse the comparison. Whether to remove equal values is a tie-policy choice; removing equals keeps the newer index, which usually expires later and is convenient.

7.3 Longest subarray with $\max - \min \leq \text{limit}$

Use one decreasing deque for the maximum and one increasing deque for the minimum. Repair while their front values differ by more than the limit.

```

package main

import "fmt"

func longestStable(a []int, limit int) int {
    maxQ, minQ := []int{}, []int{}
    left, answer := 0, 0
    for right, value := range a {
        for len(maxQ) > 0 && a[maxQ[len(maxQ)-1]] <= value {
            maxQ = maxQ[:len(maxQ)-1]
        }
        for len(minQ) > 0 && a[minQ[len(minQ)-1]] >= value {
            minQ = minQ[:len(minQ)-1]
        }
        maxQ = append(maxQ, right)
        minQ = append(minQ, right)
        for int64(a[maxQ[0]])-int64(a[minQ[0]]) > int64(limit) {
            if maxQ[0] == left {
                maxQ = maxQ[1:]
            }
            if minQ[0] == left {
                minQ = minQ[1:]
            }
            left++
        }
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    fmt.Println(longestStable([]int{8, 2, 4, 7}, 4))
}

```

7.4 Deque versus heap versus ordered structures

Structure	Update/query	Strength	Limitation
Monotonic deque	Amortized $O(1)$	Fast max/min for FIFO windows	Only extrema; careful invariant
Ordered tree	$O(\log k)$	Exact min/max and duplicates	Requires a third-party Go package
Heap + lazy deletion	$O(\log k)$	Useful for extrema/median components	Stale entries and bookkeeping
Two heaps	$O(\log k)$	Median and quantiles	Needs balance and lazy deletion

8. Ordered windows, medians, and richer state

When add/remove is easy but the statistic is not a simple aggregate, choose a data structure that supports both boundaries.

8.1 Fixed-window median with two heaps

Maintain a max-heap for the lower half and a min-heap for the upper half. The lower half has the same logical size as the upper half or one more. Go container/heap cannot delete an arbitrary outgoing value, so the complete program uses delayed deletion and separate logical sizes.

```
package main

import (
    "container/heap"
    "fmt"
)

type minHeap []int

func (h minHeap) Len() int           { return len(h) }
func (h minHeap) Less(i, j int) bool { return h[i] < h[j] }
func (h minHeap) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }
func (h *minHeap) Push(x any)        { *h = append(*h, x.(int)) }
func (h *minHeap) Pop() any {
    old := *h
    x := old[len(old)-1]
    *h = old[:len(old)-1]
    return x
}

type maxHeap []int

func (h maxHeap) Len() int           { return len(h) }
func (h maxHeap) Less(i, j int) bool { return h[i] > h[j] }
func (h maxHeap) Swap(i, j int)      { h[i], h[j] = h[j], h[i] }
func (h *maxHeap) Push(x any)        { *h = append(*h, x.(int)) }
func (h *maxHeap) Pop() any {
    old := *h
    x := old[len(old)-1]
    *h = old[:len(old)-1]
    return x
}

type dualHeap struct {
    small          maxHeap
    large          minHeap
    delayed        map[int]int
    smallSize, largeSize, k int
}

func newDualHeap(k int) *dualHeap {
    return &dualHeap{delayed: make(map[int]int), k: k}
}

func (d *dualHeap) pruneSmall() {
    for len(d.small) > 0 && d.delayed[d.small[0]] > 0 {
        value := heap.Pop(&d.small).(int)
        d.delayed[value]--
    }
}

func (d *dualHeap) pruneLarge() {
    for len(d.large) > 0 && d.delayed[d.large[0]] > 0 {
        value := heap.Pop(&d.large).(int)
        d.delayed[value]--
    }
}
```

```

}
func (d *dualHeap) balance() {
    if d.smallSize > d.largeSize+1 {
        heap.Push(&d.large, heap.Pop(&d.small))
        d.smallSize--
        d.largeSize++
        d.pruneSmall()
    } else if d.smallSize < d.largeSize {
        heap.Push(&d.small, heap.Pop(&d.large))
        d.smallSize++
        d.largeSize--
        d.pruneLarge()
    }
}
func (d *dualHeap) add(value int) {
    if len(d.small) == 0 || value <= d.small[0] {
        heap.Push(&d.small, value)
        d.smallSize++
    } else {
        heap.Push(&d.large, value)
        d.largeSize++
    }
    d.balance()
}
func (d *dualHeap) remove(value int) {
    d.delayed[value]++
    if value <= d.small[0] {
        d.smallSize--
        if value == d.small[0] {
            d.pruneSmall()
        }
    } else {
        d.largeSize--
        if len(d.large) > 0 && value == d.large[0] {
            d.pruneLarge()
        }
    }
    d.balance()
}
func (d *dualHeap) median() float64 {
    if d.k%2 == 1 {
        return float64(d.small[0])
    }
    return (float64(d.small[0]) + float64(d.large[0])) / 2
}

func medians(a []int, k int) []float64 {
    if k <= 0 || k > len(a) {
        return nil
    }
    d := newDualHeap(k)
    answer := []float64{}
    for i, value := range a {
        d.add(value)
        if i >= k {
            d.remove(a[i-k])
        }
        if i >= k-1 {
            answer = append(answer, d.median())
        }
    }
    return answer
}

func main() {
    fmt.Println(medians([]int{1, 3, -1, -3, 5, 3, 6, 7}, 3))
}

```

Each logical insertion and deletion costs $O(\log k)$, so all medians cost $O(n \log k)$. Delayed values may remain physically inside a heap until they reach its top; pruning removes them before that top is queried.

8.2 Choosing the state representation

Need	Best first choice	Notes
Small known alphabet	<code>[26]int / [256]int</code>	Fast, deterministic, cache friendly
Arbitrary sparse keys	map	Expected $O(1)$; erase zeros when needed
Ordered keys / min and max	ordered tree (third-party) or heaps	$O(\log k)$, exact order
One extreme	monotonic deque	$O(n)$ total for FIFO movement
Median	two heaps + delayed deletion	Maintain balance and ordering invariants
Bit presence for tiny universe	bitmask	Very compact; multiplicity still needs counts

8.3 Coordinate compression

If values are arbitrary but known in advance, sort unique values and map each to $[0, m)$. Then a `[]int` frequency slice can replace a map. This improves predictability and can be useful when many window updates are performed. Compression preserves equality and order, not original distances unless handled separately.

9. When ordinary sliding window fails

The most advanced skill is recognizing when the left pointer cannot make an irreversible safe decision.

9.1 Negative numbers break sum monotonicity

Consider longest subarray with sum at most S . With non-negative values, if the sum is too large, future expansion cannot fix it, so shrinking is safe. With negative values, a future negative number could restore validity while preserving the old left endpoint. Shrinking now may discard the optimal answer.

Counterexample

For $a = [5, -10, 5]$ and $S = 0$, shrinking immediately after seeing 5 discards index 0. But the full length-3 subarray has sum 0 and is valid. The classic variable sum window is therefore incorrect.

9.2 Exact sum with arbitrary integers: prefix frequencies

Let $\text{prefix}[j]$ be the sum of the first j elements. The sum of $[i, j-1]$ is $\text{prefix}[j] - \text{prefix}[i]$. It equals target when an earlier prefix is $\text{prefix}[j] - \text{target}$. Count earlier prefix values in a hash map.

```
package main

import "fmt"

func subarraySumEqualsK(a []int, target int64) int64 {
    seen := map[int64]int64{0: 1}
    var prefix, answer int64
    for _, value := range a {
        prefix += int64(value)
        answer += seen[prefix-target]
        seen[prefix]++
    }
    return answer
}

func main() {
    fmt.Println(subarraySumEqualsK([]int{1, 1, 1}, 2))
}
```

9.3 Shortest sum at least K with arbitrary integers

Use prefix sums plus a monotonic deque of candidate prefix indices. For current prefix i , if $\text{prefix}[i] - \text{prefix}[\text{dq.front}()] \geq K$, update the answer and pop front because that start cannot produce a shorter future answer than the current one. Remove from the back while its prefix sum is no smaller than the current prefix: the newer, smaller prefix dominates it.

```

package main

import "fmt"

func shortestSubarray(a []int, target int64) int {
    prefix := make([]int64, len(a)+1)
    for i, value := range a {
        prefix[i+1] = prefix[i] + int64(value)
    }
    deque := []int{}
    answer := len(a) + 1
    for i := 0; i <= len(a); i++ {
        for len(deque) > 0 && prefix[i]-prefix[deque[0]] >= target {
            if length := i - deque[0]; length < answer {
                answer = length
            }
            deque = deque[1:]
        }
        for len(deque) > 0 && prefix[deque[len(deque)-1]] >= prefix[i] {
            deque = deque[:len(deque)-1]
        }
        deque = append(deque, i)
    }
    if answer == len(a)+1 {
        return -1
    }
    return answer
}

func main() {
    fmt.Println(shortestSubarray([]int{2, -1, 2}, 3))
}

```

9.4 Other warning signs

- **Subsequence rather than substring:** contiguity is absent; consider DP, greedy, or indexed positions.
- **Range must satisfy a non-hereditary property:** removing from the left may unpredictably destroy and restore validity.
- **Need arbitrary range queries:** prefix sums, sparse tables, Fenwick or segment trees may fit better.
- **Offline reordering is allowed:** Mo's algorithm moves both boundaries and answers queries, but it is not the classic one-pass sliding window.
- **Multiple independent intervals:** interval DP or sweep-line methods may be required.

9.5 Sliding window versus related techniques

Technique	Use when	Key difference
Prefix sum	Associative range aggregate or exact sums	No validity repair; subtract prefixes
Binary search + window	Feasibility is monotone in length/value	Window checks a guessed answer
Two pointers on sorted data	Pairs/triples by value order	Pointers do not summarize a contiguous original range
Monotonic stack	Next greater/smaller, spans	One directional dominance, not a moving interval
Mo's algorithm	Many offline range queries	Boundary movement is reordered, about $O((n+q)\sqrt{n})$
DP	Overlapping states without safe boundary discard	Stores more histories instead of one interval

10. Proofs, invariants, and derivation

A template becomes reliable when you can state why every pointer movement is safe.

10.1 A proof recipe for variable windows

Step 1 - Define the state. Prove add/remove operations make it exactly describe $[left, right]$.

Step 2 - State the invariant. Example: after the repair loop, the window contains at most k distinct values.

Step 3 - Prove safe shrinking. Show that every removed start cannot lead to a better candidate for the current or any relevant future right endpoint.

Step 4 - Prove answer coverage. Associate every optimal or counted range with the moment its right endpoint is processed.

Step 5 - Prove complexity. Each boundary moves forward at most n times; multiply by the data-structure update cost.

10.2 Proof sketch: longest at most k distinct

- The frequency map and distinct counter exactly describe the active interval.
- The repair loop stops only when $distinct \leq k$.
- When left advances because $distinct > k$, the discarded window is invalid. Any extension before enough categories disappear remains invalid.
- After repair, $[left, right]$ is the longest valid suffix ending at right, so updating the global maximum examines an optimal endpoint-specific candidate.
- Both pointers only advance, and each hash update is expected $O(1)$, so total expected time is $O(n)$.

10.3 Proof sketch: monotonic maximum deque

When index i is removed from the back because $a[j] \geq a[i]$ and $j > i$, index i is dominated: in every future window containing i , j is also present until i expires, and j is at least as large. Thus i can never again be the maximum. Expired indices are removed from the front. What remains is ordered by value, so the front is the correct maximum.

10.4 Derive rather than memorize

Question to ask	Example answer
What enters?	$a[right]$ increments sum/frequency/deque.
What leaves?	$a[left]$ decrements state before left advances.
What is invalid?	$distinct > k$, $zeros > k$, $max-min > limit$.
Do I repair to valid or shrink while valid?	Depends on longest vs shortest objective.
What candidates become known now?	Best ending at right, or all valid suffixes ending at right.
What assumption makes discard safe?	Non-negativity, hereditary validity, or dominance.

11. Debugging and common mistakes

Most bugs come from a broken invariant, an off-by-one boundary, or an unstated assumption.

11.1 Off-by-one rules

- Inclusive $[left, right]$ length is $right - left + 1$.
- A fixed window becomes available when $right \geq k - 1$.
- For a new fixed window ending at $right$, the expired index is $right - k$ after adding, or anything smaller than $right - k + 1$ for a deque.
- Decide whether comparisons are strict: $sum < K$ versus $sum \leq K$, $product < K$ versus $\leq K$, and $max-min \leq limit$.

11.2 Frequent implementation failures

Bug	Symptom	Fix
Use if instead of while	Window remains invalid	Shrink until invariant holds
Update answer too early	Invalid ranges included	Record only at the proven event
Forget outgoing update	State drifts from window	Remove before incrementing left
Leave zero map entries	Wrong distinct count	Erase when frequency reaches zero
Store deque values only	Cannot expire duplicates	Store indices
Move left backward	Duplicate-window bug	Use $\max(left, last+1)$
Use int for totals	Overflow	Use int64 for sums/counts/products
Assume sum monotonic with negatives	Missed optimum	Use prefix-based method
forget heap lazy deletion	stale outgoing values remain active	track delayed counts and logical sizes
$k=0$ / $k>n$ unhandled	Undefined semantics	Validate constraints explicitly

11.3 Trace table method

For each right endpoint, write: incoming item, state after add, removals, final left, validity, and answer change. This exposes almost every pointer bug.

right	in	state after add	left removals	final $[l,r]$	answer
0	2	sum=2	-	$[0,0]$	1
1	1	sum=3	-	$[0,1]$	2
2	5	sum=8	remove 2,1	$[2,2]$	2
3	1	sum=6	-	$[2,3]$	2
4	1	sum=7	-	$[2,4]$	3

Example assumes longest sum ≤ 7 with non-negative numbers.

11.4 Edge-case test suite

- Empty input; one element; all equal; all distinct; all zeros.
- $k = 0, 1, n, n+1$; target negative/zero; no valid window; whole array valid.
- Duplicate values at both boundaries; optimum at beginning or end.
- Large values causing overflow; characters outside expected alphabet.
- For monotonic deque: strictly increasing, strictly decreasing, and all equal.

12. Go template catalog

Use these as skeletons. Replace State, validity, and answer logic only after writing the invariant.

12.1 Fixed-size template

```
package main

import "fmt"

// Fixed-window template instantiated with sum as its state.
func fixedWindow(a []int, k int) []int {
    if k <= 0 || k > len(a) {
        return nil
    }
    left, sum := 0, 0
    answer := []int{}
    for right, value := range a {
        sum += value
        if right-left+1 > k {
            sum -= a[left]
            left++
        }
        if right-left+1 == k {
            answer = append(answer, sum)
        }
    }
    return answer
}

func main() {
    fmt.Println(fixedWindow([]int{1, 2, 3, 4, 5}, 3))
}
```

12.2 Longest valid template

```
package main

import "fmt"

// Longest-valid template: at most k zeros.
func longestValid(a []int, k int) int {
    left, zeros, answer := 0, 0, 0
    for right, value := range a {
        if value == 0 {
            zeros++
        }
        for zeros > k {
            if a[left] == 0 {
                zeros--
            }
            left++
        }
        if length := right - left + 1; length > answer {
            answer = length
        }
    }
    return answer
}

func main() {
    fmt.Println(longestValid([]int{1, 0, 1, 1, 0, 1}, 1))
}
```

12.3 Shortest valid template

```
package main

import "fmt"

// Shortest-valid template: positive values with sum at least target.
func shortestValid(a []int, target int) int {
    left, sum, answer := 0, 0, len(a)+1
    for right, value := range a {
        sum += value
        for sum >= target {
            if length := right - left + 1; length < answer {
                answer = length
            }
            sum -= a[left]
            left++
        }
    }
    if answer == len(a)+1 {
        return 0
    }
    return answer
}

func main() {
    fmt.Println(shortestValid([]int{2, 3, 1, 2, 4, 3}, 7))
}
```

12.4 Count all hereditary-valid windows

```
package main

import "fmt"

// Counting template: non-negative values with sum at most limit.
func countValid(a []int, limit int) int64 {
    left, sum := 0, 0
    var answer int64
    for right, value := range a {
        sum += value
        for sum > limit {
            sum -= a[left]
            left++
        }
        answer += int64(right - left + 1)
    }
    return answer
}

func main() {
    fmt.Println(countValid([]int{1, 2, 1}, 3))
}
```

12.5 Exactly-k transform

```

package main

import "fmt"

func atMost(a []int, k int) int64 {
    if k < 0 {
        return 0
    }
    freq := make(map[int]int)
    left := 0
    var answer int64
    for right, value := range a {
        freq[value]++
        for len(freq) > k {
            outgoing := a[left]
            freq[outgoing]--
            if freq[outgoing] == 0 {
                delete(freq, outgoing)
            }
            left++
        }
        answer += int64(right - left + 1)
    }
    return answer
}

func exactlyK(a []int, k int) int64 {
    return atMost(a, k) - atMost(a, k-1)
}

func main() {
    fmt.Println(exactlyK([]int{1, 2, 1, 2, 3}, 2))
}

```

12.6 Monotonic deque template

```

package main

import "fmt"

// Monotonic-deque template instantiated for fixed-window maximum.
func windowMaximum(a []int, k int) []int {
    if k <= 0 || k > len(a) {
        return nil
    }
    deque, answer := []int{}, []int{}
    for right, value := range a {
        for len(deque) > 0 && a[deque[len(deque)-1]] <= value {
            deque = deque[:len(deque)-1]
        }
        deque = append(deque, right)
        left := right - k + 1
        if deque[0] < left {
            deque = deque[1:]
        }
        if left >= 0 {
            answer = append(answer, a[deque[0]])
        }
    }
    return answer
}

func main() {
    fmt.Println(windowMaximum([]int{1, 3, -1, -3, 5}, 3))
}

```

12.7 Binary search on answer plus fixed window

Sometimes the question asks for the maximum feasible length and feasibility is monotone by length. Binary-search the length, and test each length with a fixed window. This often costs $O(n \log n)$. Before using it, check whether a direct variable window can already solve the problem in $O(n)$.

13. Advanced patterns and extensions

Sliding window is a framework: boundary movement can be combined with transformations, dequeues, hashing, and binary search.

13.1 Complement window: take from both ends

If you must take exactly k items from the two ends of an array, choosing those items is equivalent to leaving one contiguous middle window of length $n-k$. Maximize taken sum by minimizing the sum of that fixed middle window. This complement transformation turns an awkward two-ended choice into a window.

13.2 Circular arrays

A circular window of length at most n can be processed over indices $0..2n-1$ using $a[i \% n]$, while preventing length from exceeding n . For fixed $k \leq n$, scan only enough doubled positions to enumerate n starting points. Avoid physically duplicating a large array unless clarity matters more than memory.

13.3 Multiple constraints

Combine state components: sum plus distinct count, max/min dequeues plus a budget, or several character requirements. The invalid predicate becomes an OR of violations. Ordinary variable windows remain valid only if shrinking moves every violated constraint monotonically toward validity.

13.4 Two windows in one pass for exactly- k counting

Instead of calling `atMost` twice, maintain two left boundaries: one for at most k and one for at most $k-1$. At each right, the number of exactly- k ranges ending there is `leftKMinus1 - leftK`. This can reduce constant factors but is harder to reason about; the two-pass identity is usually cleaner.

13.5 Frequency of the maximum element

Problems may ask for subarrays where the global maximum appears at least k times. Once the active window contains k copies, every extension of its left boundary in the opposite direction may be counted. Carefully identify whether you are counting valid prefixes or valid suffixes; not every counting problem uses `right-left+1`. Derive the combinatorics from the endpoint fixed by the loop.

13.6 Windows over streams

A fixed window naturally supports streaming: keep only the last k items or the state needed to expire them. Time-based windows require timestamps and expire all events older than the cutoff. Out-of-order events need watermark/buffering semantics and go beyond the simple array model.

13.7 2D sliding windows

For matrix rectangle sums, a 2D prefix sum usually gives $O(1)$ per rectangle. For row-wise/column-wise extrema, apply monotonic dequeues in two passes. A k by k maximum filter can compute row maxima of width k , then column maxima of height k , in $O(\text{rows} * \text{cols})$.

13.8 Smallest range covering k sorted lists

Merge all values with their list ID, sort them, then find the smallest window containing every list ID. This is minimum-window substring generalized from characters to categories. A heap-based k-way merge is another approach.

13.9 Offline queries and Mo's algorithm

Mo's algorithm orders arbitrary range queries so consecutive queries require few add/remove operations. Unlike the classic pattern, both boundaries may move in both directions, and correctness does not rely on monotone validity. It is useful when updates are cheap but no fast per-query data structure exists.

14. Worked pattern map

Different stories often reduce to the same invariant.

Problem phrase	Invariant / transformation	Answer event
Maximum average length k	Fixed size; rolling sum	When size == k
Longest no repeats	Every frequency ≤ 1	After repair
Longest at most k distinct	distinct $\leq k$	After repair
Shortest covering pattern	missing == 0	While valid
Permutation in string	Fixed size + matching frequencies	When size == p
Count exactly k odds	atMost(k) - atMost(k-1)	Add valid suffix count
Max of every length k	Decreasing index deque	When size == k
Longest max-min $\leq L$	Two monotonic deque	After repair
Exact sum, arbitrary ints	Prefix frequency map	At each prefix
Shortest sum $\geq K$, arbitrary ints	Prefix + monotonic deque	Pop valid fronts
Take k cards from ends	Minimize complement window n-k	When size == n-k
Cover k sorted lists	Window over merged (value, list)	While all lists present

14.1 Interview communication script

- State the brute force and its complexity.
- Point out the overlap between neighboring contiguous ranges.
- Define [left, right], the maintained state, and the validity predicate.
- Explain exactly when right expands and when left shrinks.
- State the invariant before coding.
- After coding, justify correctness, $O(n)$ amortized movement, update cost, and space.
- Name assumptions explicitly: non-negative values, alphabet, k bounds, overflow type.

15. Graded practice set

Solve in order. For each problem, first write the family, state, invalid predicate, answer event, and complexity without code.

15.1 Basic

#	Problem	Main lesson
1	Maximum sum subarray of size k	Rolling fixed state
2	Average of every subarray of size k	Same state, different output
3	First negative in every window	Queue indices and expiration
4	Count distinct in each window	Frequency map lifecycle
5	Maximum number of vowels in length k substring	Fixed violation counter
6	Find all anagrams	Fixed frequency matching

15.2 Intermediate

#	Problem	Main lesson
7	Longest substring without repeats	Repair duplicates
8	Longest subarray with at most k zeros	Budget invariant
9	Fruit into baskets / at most two types	At most k distinct
10	Minimum size sum $\geq S$, positive array	Shrink while valid
11	Minimum window substring	Multiplicity and matched count
12	Count product $< k$, positive array	Count valid suffixes
13	Count exactly k distinct	Difference of at-most counts
14	Binary subarrays with sum goal	Exactly via at-most transform
15	Longest repeating character replacement	Dominant frequency

15.3 Advanced

#	Problem	Main lesson
16	Sliding window maximum	Monotonic deque
17	Longest range with max-min $\leq$ limit	Two deques
18	Sliding window median	Two heaps plus lazy deletion
19	Shortest subarray sum $\geq K$ with negatives	Prefix monotonic deque
20	Smallest range covering k lists	Merged categorical window
21	Subarrays with exactly k odd numbers	Exactly transform
22	Maximum points from taking k ends	Complement window
23	Substring concatenation of all words	Multiple offset windows
24	Minimum operations to reduce x to zero	Longest complement sum
25	Count complete subarrays	At-most or ending-point count

15.4 Expert extensions

#	Problem	Direction
26	2D k x k maximum filter	Two-pass monotonic deques
27	Time-based event-rate window	Timestamp deque and expiry
28	Median absolute deviation in fixed windows	Ordered partitions + sums
29	Many offline distinct-range queries	Mo's algorithm
30	Design a generic reusable window aggregator	Interfaces, invariants, tests

16. Hints and compact solutions

Use these only after attempting the classification and invariant yourself.

16.1 Problems 1-6

1-2. Initialize the first k sum or use the universal fixed template. For averages, divide only when emitting. **3.** Store negative indices; expire indices smaller than left. **4.** Increment incoming, decrement outgoing, erase zero. **5.** Track a vowel count. **6.** Maintain a difference array or missing-instance counter over exact-size windows.

16.2 Problems 7-15

7. Frequency repair or last-seen jump. **8.** Invalid when zeros $> k$. **9.** Invalid when map size > 2 . **10.** Record while sum $\geq S$, then shrink; positivity is required. **11.** Count missing required instances. **12.** Repair product $\geq k$ and add window length. **13.** $\text{atMost}(k) - \text{atMost}(k-1)$. **14.** Same transform because binary values are non-negative. **15.** replacements = length - largest frequency.

16.3 Problems 16-25

16. Decreasing deque of indices. **17.** Decreasing max deque plus increasing min deque. **18.** Two heaps with lazy deletion. **19.** Prefix sums in an increasing deque. **20.** Sort merged (value,listID) pairs, then cover all IDs. **21.** Treat odd as 1, even as 0; use at-most difference. **22.** total minus minimum length $n-k$ middle sum. **23.** Run one word-aligned window per offset modulo word length. **24.** Find longest subarray with sum total- x ; positivity enables a normal window. **25.** Count windows containing all globally distinct values by shrinking while complete and counting possible starts.

16.4 Problems 26-30

26. Compute width- k row maxima, then height- k column maxima. **27.** Expire timestamps earlier than now-duration. Define inclusive cutoff precisely. **28.** Median minimizes absolute deviation; maintain two ordered halves plus sums of each half. **29.** Sort queries by blocks and update frequency state as boundaries move. **30.** Expose add/remove/query, document valid movement directions, and property-test against brute force.

16.5 A reusable brute-force oracle

For small random arrays, compare your optimized result with an obviously correct $O(n^2)$ implementation. This is one of the fastest ways to validate window invariants and boundary conditions.

```
package main

import "fmt"

// Brute-force oracle: longest subarray with sum <= limit.
func brute(a []int, limit int64) int {
    answer := 0
    for left := 0; left < len(a); left++ {
        var sum int64
        for right := left; right < len(a); right++ {
            sum += int64(a[right])
            if sum <= limit && right-left+1 > answer {
                answer = right - left + 1
            }
        }
    }
    return answer
}

func main() {
    fmt.Println(brute([]int{2, 1, 5, 1, 1}, 7))
}
```

17. Final cheat sheet

The whole pattern, compressed into one reference page.

Recognition

Contiguous range + overlapping candidates + cheap add/remove + safe monotone boundary movement. If any part is missing, reconsider the technique.

Core formulas

Idea	Formula / rule
Inclusive length	$\text{right} - \text{left} + 1$
Valid suffixes ending at right	$\text{right} - \text{left} + 1$
Exactly k	$\text{atMost}(k) - \text{atMost}(k - 1)$
Fixed outgoing index after adding right	$\text{right} - k$
Replacement cost	window length - maximum frequency
Subarray sum by prefixes	$\text{prefix}[\text{r} + 1] - \text{prefix}[\text{l}]$
All subarrays count	$n(n + 1) / 2$

Answer-event map

Goal	When to update
Best exact length k	After the active size becomes k
Longest under an upper bound	After repairing invalidity
Shortest reaching a lower bound	Inside the while-valid shrink loop
Count hereditary-valid ranges	After repair: add current window length
Minimum cover	Inside the complete-cover shrink loop

Complexity

Classic window: $O(n)$ boundary movements. With state update U: $O(nU)$. Hash/array state usually $O(n)$ total; ordered state $O(n \log k)$; monotonic deque $O(n)$ amortized. Space ranges from $O(1)$ to $O(k)$ or $O(\text{alphabet})$.

The one sentence to remember

Expand to explore, shrink to restore or optimize, maintain only what the active interval needs, and never discard a boundary unless you can prove it cannot matter later.

Mastery test

You have mastered sliding window when you can: recognize it without keywords; derive rather than memorize the loop; state the invariant; explain answer placement; prove pointer movements; identify negative-number failures; choose the right state structure; and reduce a new story to a known window family.

Seven-day mastery plan

Day	Focus	Deliverable
1	Core idea + fixed windows	Solve practice 1-5 and trace two by hand
2	Variable longest/shortest	Solve 7-10; write the invariant first
3	String frequency state	Solve 6, 11, and 15 without copying templates
4	Counting transformations	Solve 12-14 and explain right-left+1
5	Monotonic dequeues	Solve 16-17; prove every deque pop is safe
6	Failure cases + advanced state	Solve 18-20 and compare alternatives
7	Mixed timed revision	Pick six unseen problems; classify before coding

Before submitting any window solution

- Check empty input, k bounds, strict versus non-strict comparisons, and overflow.
- Verify state exactly matches [left, right] after every add and remove.
- Confirm the answer line occurs only when its candidate is mathematically known.
- State the assumption that makes moving left irreversible and safe.
- Test against a brute-force oracle on small random and adversarial inputs.